

AOIA-Core Runtime Architecture Report for Bash/Shell Module Review

1. Executive Summary

AOIA-Core is a local-first deterministic epistemic-control runtime focused on inspectable, classifiable, and auditable AI-agent actions before human-approved execution.

This report is specifically for planning the next Bash/Shell module after GT-RUNTIME-6. It summarizes the current runtime architecture, existing shell-safety state, relevant execution boundaries, and the smallest safe next step. It does not implement Bash/Shell execution and does not claim that AOIA-Core is a complete security sandbox.

2. Current Repository State

- Repository name: AOIA-Core
- Branch: `dev/gt-runtime-5-single-event-ledger`
- Latest commit inspected: `c0aa676 feat: add GT-RUNTIME-6 shell safety metrics harness`
- Working tree status at start of this report step: no tracked source modifications; two prior untracked post-GT-RUNTIME-6 audit report files were present from the previous PDF baseline step
- Stash status summary: `stash@{0}` contains Cloudflare WIP preserved before this report step
- Cloudflare WIP: preserved in stash and outside this report scope
- GT-RUNTIME-6 status: implemented, committed, pushed, and validated
- Test status from the post-GT-RUNTIME-6 validation run: PASS, 372 tests, 4 skipped
- Validator status from GT-RUNTIME-6: PASS, 12 ledger events

No Cloudflare files were restored or inspected from the stash for this report.

3. Runtime Directory Structure

Readable runtime-relevant structure:

```
runtime/  
  main.py  
  webapp.py  
  runtime_paths.py  
  requirements.txt  
  adaptive_routing/  
  commands/  
  knowledge/  
    bash/
```

```

    canonical/
    grammar/
    validator/
    tools/
memory/
memory_hats/
orchestrator/
providers/
retrieval/
router/
tools/
    command_grammar.py
    command_grammar_cli.py
    event_ledger.py
    executor.py
    provenance.py
    rhcsa_search.py
    shell_tools.py
    validator.py

tests/
    test_command_grammar.py
    test_command_grammar_cli.py
    test_event_ledger.py
    test_executor_containment.py
    test_gt_runtime_6_safety_metrics.py
    test_respond_shell_safety.py
    test_runtime_router_contract_guard.py
    plus other runtime, provenance, retrieval, and memory tests

tools/
    validate_safety.py

corpus/
    shell_cases.jsonl

docs/audit/
    GT_RUNTIME_1_FIX_BOOT_BLOCKERS_REPORT_01_JUNE_2026.md
    GT_RUNTIME_2_MOVE_GENERATED_STATE_OUT_OF_REPO_REPORT_01_JUNE_2026.md
    GT_RUNTIME_3_RESPOND_MESSAGE_SHELL_SAFETY_FILTER_REPORT_01_JUNE_2026.md
    GT_RUNTIME_4_SHELL_ADVICE_APPROVAL_WARNING_GATE_REPORT_01_JUNE_2026.md
    GT_RUNTIME_5_SINGLE_EVENT_LEDGER_PROTOTYPE_REPORT_01_JUNE_2026.md
    GT_RUNTIME_6_SHELL_SAFETY_METRICS_HARNESS_REPORT_02_JUNE_2026.md
    GT_RUNTIME_FULL_CLOSURE_REPORT_01_JUNE_2026.md
    AOIA_CORE_POST_GT_RUNTIME_6_EXTERNAL_AUDIT_BASELINE_02_JUNE_2026.md
    AOIA_CORE_RUNTIME_ARCHITECTURE_FOR_BASH_MODULE_REVIEW_02_JUNE_2026.md

```

The full repository contains more knowledge, report, runtime-state, and test files. This report intentionally lists only files relevant to Bash/Shell module planning.

4. Runtime Components

Main runtime components confirmed from current inspection:

- `runtime/main.py`: main agent runtime loop. It builds prompts, coordinates model planning, routes local knowledge, validates structured actions, invokes the execution engine, and records operational history.
- `runtime/tools/executor.py`: dispatches structured tool actions. It contains the `ExecutionEngine`, a tool registry, approval prompting, shell action handling, filesystem/browser dispatch, and operational event recording.
- `runtime/tools/shell_tools.py`: execution-capable shell helper. It uses `subprocess.run()` with `bash -lc`, supports captured and interactive modes, and returns stdout/stderr/exit metadata. This is a high-caution integration point for any future Bash module.
- `runtime/tools/validator.py`: validates model-generated structured actions, blocks selected shell patterns, classifies shell commands into permission modes, and inspects respond-message shell advice for unsafe/high-risk text.
- `runtime/tools/command_grammar.py`: deterministic advisory command-shape validator. It is documented as local, read-only, non-executing, and not a replacement for `ShellCheck` or a shell AST parser.
- `runtime/tools/command_grammar_cli.py`: CLI wrapper for command grammar classification.
- `runtime/tools/event_ledger.py`: append-only event ledger prototype with hash chaining, timezone-aware UTC timestamps, payload redaction for obvious secret keys, and allowed event types including shell-safety events.
- `runtime/tools/provenance.py` and `runtime/tools/provenance_readout.py`: append-only provenance store and readout/integrity reporting utilities.
- `runtime/adaptive_routing/epistemic_kernel.py`: deterministic epistemic routing and provenance attachment around local knowledge retrieval.
- `runtime/retrieval/`: local Linux/RHCSA retrieval facade and scoring modules.
- `runtime/orchestrator/knowledge_router.py`: knowledge routing coordination.
- `runtime/providers/`: provider configuration and cloud-model adapters. Provider routing is outside the scope of this report and was not modified.
- `runtime/memory/` and `runtime/memory_hats/`: local memory, history, evidence, and advisory tag support.
- `tools/validate_safety.py`: GT-RUNTIME-6 standalone shell-safety metrics harness.
- `corpus/shell_cases.jsonl`: controlled JSONL corpus for GT-

RUNTIME-6.

Not confirmed from current inspection:

- A full Bash execution sandbox.
- A shell AST parser.
- A complete Bash/Shell module with rollback, TTY/session management, or filesystem impact preview.

5. Dependencies and Python Environment

Visible dependency file:

`runtime/requirements.txt`

Visible dependencies declared there:

```
google-genai>=1.0.0
playwright>=1.59.0
beautifulsoup4>=4.12.0
rich>=13.7.0
textual>=0.86.0
```

No root-level `requirements.txt`, `pyproject.toml`, `setup.py`, `setup.cfg`, `Pipfile`, or `poetry.lock` was found during the safe inspection.

Observed standard-library usage relevant to Bash/Shell planning includes:

- `subprocess`: used by `runtime/tools/shell_tools.py`, `runtime/commands/local_commands.py`, `runtime/tools/build_rhcsa_library.py`, and test modules.
- `os`: used for environment variables and runtime state in `runtime/main.py`, `runtime/providers/`, `runtime/runtime_paths.py`, and utility modules.
- `pathlib`: used broadly for path-safe runtime state, memory, provenance, and tooling.
- `json`: used broadly for structured runtime state, tool actions, metrics, ledgers, and knowledge data.
- `argparse`: used in CLI utilities such as `tools/validate_safety.py`, `runtime/tools/rhcsa_search.py`, `runtime/tools/provenance_readout.py`, and repository guard tools.
- `re`: used for routing, command grammar, shell-safety pattern matching, and ledger validation.
- `time` and `datetime`: used for timing, timestamps, runtime logs, and ledger/provenance records.
- `shlex`: used in `runtime/tools/command_grammar.py` for command-shape parsing.

This inspection did not install dependencies and did not use network access.

6. Configuration Files and Environment Boundaries

Configuration files and state paths observed safely:

- runtime/adaptive_routing/aoia_config.json
- runtime/adaptive_routing/routing_modes.json
- runtime/state/model_config.json
- runtime/state/providers.json
- runtime/prompts/system_prompt.txt
- runtime/requirements.txt
- Local secret file candidates under user config directories are referenced by code, but their contents were not inspected.

Raw .env contents were not inspected. No secrets, API keys, tokens, private keys, or credentials are included in this report.

Environment variable names observed from source code, with values intentionally redacted:

```
AGENT_DEBUG=REDACTED
EPISTEMIC_KILL_SWITCH=REDACTED
EPISTEMIC_DISABLE_MODEL=REDACTED
EPISTEMIC_DISABLE_KNOWLEDGE_ROUTE=REDACTED
EPISTEMIC_DISABLE_MEMORY_HATS=REDACTED
EPISTEMIC_DISABLE_REASONING_TRACE=REDACTED
EPISTEMIC_DISABLE_UNKNOWN_FALLBACK=REDACTED
GEMINI_API_KEY=REDACTED
GOOGLE_API_KEY=REDACTED
OPENROUTER_API_KEY=REDACTED
DEEPSEEK_API_KEY=REDACTED
DEEPSEEK_BASE_URL=REDACTED
XAI_API_KEY=REDACTED
XAI_BASE_URL=REDACTED
AUREON_API_BASE_URL=REDACTED
AUREON_API_KEY=REDACTED
OLLAMA_BASE_URL=REDACTED
HF_TOKEN=REDACTED
HUGGINGFACE_API_KEY=REDACTED
GEMMA_HF_MODEL=REDACTED
GEMMA_OPENAI_BASE_URL=REDACTED
GEMMA_OPENAI_API_KEY=REDACTED
OPENAI_COMPATIBLE_MAX_TOKENS=REDACTED
USER=REDACTED
```

7. Runtime Entrypoints

Confirmed or likely runtime entrypoints:

Path	Purpose	CLI behavior	Shell/system execution	Safety notes
runtime/main.py	Main local agent runtime loop	Interactive runtime behavior is present	Indirectly execution-capable through ExecutionEngine	Requires action validation and human approval for non-respond actions that require confirmation
runtime/webapp.py	Web entrypoint	Has main() and if __name__ == "__main__"	Not confirmed as shell-execution-capable from inspected excerpt	Web surface should stay separate from execution policy
runtime/run.sh	Shell launcher	Shell script entrypoint	Execution-capable as a launcher	Not expanded in this report beyond path-level identification
runtime/run_web.sh	Web launcher	Shell script entrypoint	Execution-capable as a launcher	Not expanded in this report beyond path-level identification
runtime/tools/Command_classifier_cli.py	Command grammar classification	Yes	Non-executing classifier from inspected source	Useful as a safe pre-execution analysis component
runtime/tools/rhcsa_search.py	Deterministic RHCSA knowledge search	Yes	Not primarily shell execution	Read-only retrieval-oriented CLI
runtime/tools/provenance_report.py	Provenance chain integrity report	Yes	No shell execution confirmed	Read-only verification/reporting

Path	Purpose	CLI behavior	Shell/system execution	Safety notes
<code>runtime/knowledge/ledger/validator.py</code>	Knowledge pack validator	No	No shell execution confirmed	Deterministic local validation
<code>tools/validate_safety.py</code>	Classifies GT-RUNTIME-6 shell-safety metrics harness	Yes	Does not execute corpus commands	Classifies strings and writes metrics/ledger artifacts

Execution-capable file confirmed:

- `runtime/tools/shell_tools.py`: uses `subprocess.run(["bash", "-lc", command], ...)`. A future Bash/Shell module must treat this as a controlled later-stage execution sink, not as the place where command proposals enter the system.

8. Existing Shell Safety State

GT-RUNTIME-6 consists of:

- `tools/validate_safety.py`
- `corpus/shell_cases.jsonl`
- `tests/test_gt_runtime_6_safety_metrics.py`
- `docs/audit/GT_RUNTIME_6_SHELL_SAFETY_METRICS_HARNESS_REPORT_02_JUNE_2026.md`
- `metrics_report.json` generated during validation
- `event_ledger.ndjson` generated during validation

The harness classifies command strings only. The corpus commands are inert test data. The harness does not execute shell commands.

The current harness is rule/regex-based unless future evidence proves otherwise.

Latest known GT-RUNTIME-6 metrics:

```

detection_rate: 1.0
false_positive_rate: 0.0
missed_dangerous: 0
warning_rate: 1.0
ledger_coverage: 1.0

```

These metrics are from a small controlled corpus and should not be presented as proof of complete real-world shell safety.

9. Bash/Shell Module Readiness Analysis

9.1 Safe integration points

Likely safe places for a future Bash/Shell module:

- Command proposal intake: before `ExecutionEngine.execute()` dispatches a `shell_execute` action.
- Pre-execution classification: reuse `tools/validate_safety.py` and related classifiers as read-only string classifiers only; do not add execution capability to the classification layer.
- Human approval gate: keep approval before any `shell_execute` path and make approval visible in the ledger.
- Event ledger writing: use `runtime/tools/event_ledger.py` for structured events such as command proposed, classification decision, approval decision, and execution result if a later execution layer is added.
- Dry-run mode: add a schema and tests for proposed command analysis without execution.
- Allowlist/denylist rules: define rules before enabling any execution path.
- Audit report generation: keep reports under `docs/audit/` and runtime-generated state outside source directories.

9.2 Risky integration points

Places that need caution:

- `runtime/tools/shell_tools.py`, because it executes through `bash -lc`.
- Any `subprocess.run()` path, including test utilities and library builders.
- Any future direct `shell=True` use. Current planning should avoid introducing it.
- Provider-generated command suggestions, especially when they appear in normal response text rather than structured actions.
- Filesystem writes, moves, and deletes in `runtime/tools/executor.py`.
- Destructive command families, service/system commands, privilege escalation, and network pipe-to-shell patterns.
- Browser and web surfaces if future UI actions can trigger shell proposals.
- Runtime logs and ledgers if they accidentally store secrets or private local file contents.

9.3 Required permissions model

Minimal permission model recommended for Bash/Shell planning:

- Read-only by default.
- Dry-run first.
- Explicit user approval before execution.
- No sudo automation in v0.1.
- No destructive execution in v0.1.
- No automatic remote code execution.

- Log every proposed command.
- Classify before any execution path exists.

9.4 Data flow proposal

Safe data flow proposal:

```
AI/model output
-> command extraction
-> normalization
-> safety classification
-> rule/reason assignment
-> ledger event
-> human approval
-> optional execution layer later
```

This report does not implement the data flow. It is a planning boundary for external review.

10. Gaps Before Bash Module Implementation

Known gaps before implementing a Bash/Shell module:

- No full adversarial shell corpus yet.
- No shell AST/parser yet.
- No complete rule ID/reason ID schema in the runtime ledger for shell classification decisions.
- No formal allowlist policy yet.
- No sandbox execution layer yet.
- No TTY/session manager yet.
- No sudo policy.
- No command rollback model.
- No filesystem impact preview.
- No large external benchmark yet.
- No formal verification yet.

11. Recommended Next Step

Smallest safe next step:

GT-RUNTIME-7: Bash/Shell Safety Planning v0.1

GT-RUNTIME-7 should not implement full Bash execution yet.

Recommended GT-RUNTIME-7 scope:

- Benchmark limitations document.
- Adversarial corpus v0.2 plan.
- Ledger schema proposal with fields:
 - classifier_version

- rule_id
- reason
- normalized_command
- risk_level
- decision
- timestamp
- Dry-run-only command proposal schema.
- Integration test plan for respond-message safety path.
- No command execution yet.

12. External Audit Questions

1. Is the proposed Bash module boundary safe enough?
2. Where should AOIA-Core place the human approval gate?
3. Should GT-RUNTIME-7 add execution, or remain dry-run only?
4. What should the minimum shell ledger schema include?
5. What shell commands should be included in adversarial corpus v0.2?
6. What would a skeptical security reviewer criticize first?
7. Is the current GT-RUNTIME-6 benchmark enough for a grant artifact?
8. What is the safest next implementation step?
9. Does this report overclaim any current capability?
10. What should be explicitly excluded from the Bash/Shell module v0.1?

13. Honest Claim Boundary

AOIA-Core currently has a reproducible local shell-safety metrics harness for controlled pre-execution classification tests. It does not yet provide complete real-world protection against all dangerous AI-generated shell commands. The next Bash/Shell module should begin with dry-run planning, schema design, adversarial corpus design, and audit integration before any execution layer is added.

14. Final Status

- Ready for external architecture audit: yes. No external audit has been performed yet.
- Ready for Bash module planning: yes.
- Ready for Bash execution implementation: no.
- Recommended next step: GT-RUNTIME-7 Bash/Shell Safety Planning v0.1, dry-run and schema only.
- PDF created: yes, by local PDF export during this report step.